

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..30

STATIC FILES

 Copy as Markdown

Static files

The `StaticRoute` class serves files from disk: images, CSS, JavaScript, fonts, or any other asset, with automatic content-type detection for common file formats. This page covers serving directories and single files, cache headers, and cache busting.

Serving static files

The simplest way to serve static files is to use `StaticRoute.directory()`:

```
final staticDir = Directory('web/static');

pod.webServer.addRoute(
  StaticRoute.directory(staticDir),
  '/static/',
);
```

This serves all files from the `web/static` directory at the `/static` path. For example, `web/static/logo.png` becomes accessible at `/static/logo.png`.

To serve one specific file instead of a directory, use

`StaticRoute.file(File('web/pages/landing.html'))`.

INFO

The `StaticRoute.directory()` factory automatically handles tail matching, so you don't need to add `**` to the path. The route will serve all files under the given prefix.

Cache control



By default, `StaticRoute` sends no `Cache-Control` header at all. Responses still carry `ETag` and `Last-Modified` headers, so browsers revalidate with conditional requests and receive a `304 Not Modified` when the file is unchanged.

To set a caching policy, pass one of the cache-control helpers as the `cacheControlFactory` parameter:

Helper	Resulting header	Use when
<code>StaticRoute.public(maxAge: ...)</code>	<code>public, max-age=N</code>	Assets that change occasionally. Caches keep them for the given lifetime
<code>StaticRoute.publicImmutable(maxAge: ...)</code>	<code>immutable, public, max-age=N</code>	Cache-busted assets whose URL changes with the content. Use a long lifetime
<code>StaticRoute.privateNoCache()</code>	<code>no-cache, private</code>	Per-user content that must be revalidated on every request
<code>StaticRoute.noStore()</code>	<code>no-store</code>	Content that must never be cached

```
pod.webServer.addRoute(  
  StaticRoute.directory(  
    staticDir,  
    cacheControlFactory: StaticRoute.public(maxAge: const Duration(minutes: 5)),  
  ),  
  '/static/',  
);
```

To set the header per environment instead of in code, set

`SERVERPOD_WEB_SERVER_STATIC_CACHE_CONTROL` to the `Cache-Control` value the route should send:

```
SERVERPOD_WEB_SERVER_STATIC_CACHE_CONTROL="public, max-age=3600"
```

The environment variable applies to every `StaticRoute` that does not pass a `cacheControlFactory` of its own.

Reserve `publicImmutable` for `cache-busted` assets: `immutable` tells caches to never revalidate, which is only safe when a changed file also gets a new URL.

On [Serverpod Cloud](#), a CDN sits in front of the web server and honors these headers, and its cache is cleared on every deploy.

Static file cache-busting

When deploying static assets, browsers and CDNs cache files aggressively for performance. This means updated files may not be served to users unless you implement a cache-busting strategy.

Relic provides `CacheBustingConfig` to automatically version your static files. For more details, see the [Relic documentation](#).

```
final staticDir = Directory('web/static');

final cacheBustingConfig = CacheBustingConfig(
  mountPrefix: '/static',
  fileSystemRoot: staticDir,
  separator: '@', // or use custom separator like '___'
);

pod.webServer.addRoute(
  StaticRoute.directory(
    staticDir,
    cacheBustingConfig: cacheBustingConfig,
    cacheControlFactory: StaticRoute.publicImmutable(maxAge: const Duration(days: 365)),
  ),
  '/static/',
);
```

With cache busting, a changed file gets a new URL, so caches can safely keep each version for a long time. This is the setup where `publicImmutable` with a long lifetime belongs.

Generating versioned URLs

Use the `assetPath()` method to generate cache-busted URLs for your assets:

```
// In your route handler
final imageUrl = await cacheBustingConfig.assetPath('/static/logo.png');
// Returns: /static/logo@<hash>.png

// Pass to your template
return MyPageWidget(logoUrl: imageUrl);
```

The cache-busting system:

- Automatically generates content-based hashes for asset versioning
- Allows custom separators (default `@`, but you can use `___` or any other)
- Preserves file extensions
- Works transparently - requesting `/static/logo@abc123.png` serves `/static/logo.png`

Conditional requests (ETags and Last-Modified)

Every `StaticRoute` automatically supports HTTP conditional requests through Relic's `StaticHandler`, along with `Range` requests for partial downloads. This provides efficient caching without transferring file content when unchanged:

Supported features:

- **ETag headers** - Content-based fingerprinting for cache validation
- **Last-Modified headers** - Timestamp-based cache validation
- **If-None-Match** - Client sends ETag, server returns 304 Not Modified if unchanged
- **If-Modified-Since** - Client sends timestamp, server returns 304 if not modified

These work automatically without configuration:

Initial request:

```
GET /static/logo.png HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
ETag: "abc123"
Last-Modified: Tue, 15 Nov 2024 12:00:00 GMT
Content-Length: 12345

[file content]
```

Subsequent request with ETag:

```
GET /static/logo.png HTTP/1.1
Host: example.com
If-None-Match: "abc123"

HTTP/1.1 304 Not Modified
ETag: "abc123"

[no body - saves bandwidth]
```

Conditional requests do the caching work in the default setup and with `public(maxAge:)`, where clients revalidate once a file's lifetime expires. With cache-busted `immutable` assets, clients skip revalidation entirely by design, since a changed file gets a new URL instead.

For dynamic content that changes per request, see [Server-side HTML](#).